

Using Large Language Models to Extract UML Class Diagrams from Java Programs

1st Hanan Abdulwahab Siala
Informatics Department
King's College London
London, UK
hanan.siala@kcl.ac.uk

2nd Kevin Lano
Informatics Department
King's College London
London, UK
kevin.lano@kcl.ac.uk

Abstract—Many organizations rely on software systems to perform their core business operations. These systems often require modernization to accommodate new requirements and demands over time. Visual representations and diagrams help maintainers to understand these systems and to assist their maintenance and evolution. Reverse engineering is used to extract different representations of software systems, and the model-driven engineering (MDE) approach can be used to assist the reverse engineering process, leading to model-driven reverse engineering (MDRE). Large Language Models (LLMs) are an advanced deep learning (DL) technology increasingly being applied in various domains. Despite this, the use of LLMs to extract diagrams from source code has rarely been used. In this paper, we introduce a novel MDRE framework, LLM4Models, that extracts UML class diagrams from Java source code files using LLMs. The paper also introduces a set of rules to extract UML class diagram representations. These rules have been implemented to create training data samples, which are then used to fine-tune the pre-trained LLM. Finally, the proposed framework is explained with an illustrative example and then evaluated.

Index Terms—Model-driven Reverse Engineering (MDRE), Unified Modeling Language (UML), Class Diagram, Machine Learning, Large language models (LLMs), Java programs, Legacy systems.

I. INTRODUCTION

Many organizations depend on software systems for their essential daily tasks. These systems are called *legacy systems* if they use outdated programming languages or platforms. Over time, these systems need to meet new requirements and business needs. Organizations face the dilemma of building new software systems or maintaining and evolving existing ones. Replacing these systems would be very expensive and time-consuming, and making changes to the existing systems would be the preferable solution.

The reverse engineering process is used to understand software systems in order to apply the required changes. Model-Driven Engineering (MDE) [1] facilitates the reverse engineering process by providing high-level abstractions of the software systems. Various models are used to represent and document software systems in Model-Driven Reverse Engineering (MDRE) approaches. The Unified Modeling Language (UML) [2] is widely used to model software systems, and it consists of several diagrams, each serving a specific purpose.

The class diagram is the most commonly used type of UML diagram, and it defines a data-oriented view of a system, focusing on the classes and their relationships within the software system. Large Language Models (LLMs) are a type of machine learning (ML) technology that are trained on a massive amount of textual data and can perform a variety of specific downstream tasks. They have had a significant impact on various domains, including software engineering, where they are used in various activities, including software development, verification, maintenance, and quality assurance [3]. Since LLMs are trained on datasets involving programming languages, they can understand the syntax and semantics of various programs, saving developers time and effort compared to traditional MDRE tools. Although combining traditional MDRE techniques with LLMs can offer many prospective benefits, research on this area remains scarce. As a result, we aim to introduce an MDRE approach to extract UML class diagrams from Java programs using LLMs. The Java language was chosen for this work because it is the most widely used language by various reverse engineering approaches [4].

The paper is organized as follows: [Section II](#) describes the proposed approach, while an illustrative example is given in [Section III](#). The evaluation of the proposed approach is presented in [Section IV](#). Related works are given in [Section V](#), while [Section VI](#) concludes the paper and presents the future work.

II. LLM4MODELS APPROACH

In this section, we introduce our MDRE approach for extracting UML class diagram representations from Java programs. In addition, we identify the rules needed to identify UML class diagram representations from Java code.

A. Approach Overview

[Fig. 1](#) shows the overall approach, and the description of its stages is as follows:

1) Constructing the fine-tuned LLM:

a) Creating a Training Dataset:

To fine-tune a pre-trained LLM, a training dataset is required, containing Java programs and their corresponding UML class diagrams. The Java programs for our dataset are taken from

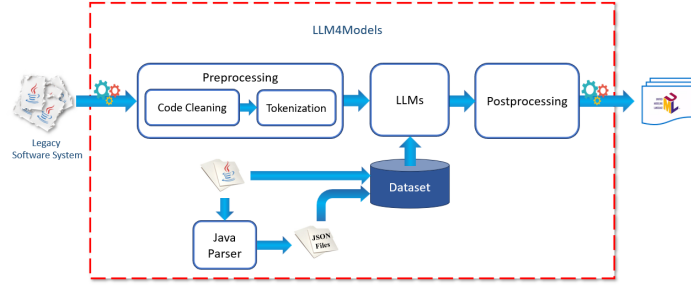


Fig. 1. LLM4Models approach overview.

various sources. For one set of cases, the AgileUML toolset [5] is utilized to randomly construct Java programs with different structures. In addition, Java programs from the AVATAR [6] dataset are selected. Approximately 1000 example programs are used for our dataset. Corresponding class diagrams are generated from the Java codes, using the abstraction rules of Section II-B. Each UML class diagram is represented by two JSON files. The first represents UML class diagram elements, while the second represents relationships between classes/interfaces. A specific parser and analyser, named *Java2JSON*, was designed and implemented to extract these representations from any Java programs using the abstraction rules.

b) Fine-tuning the Pre-trained LLM:

In our approach, a pre-trained LLM is fine-tuned using Supervised fine-tuning (SFT). Quantized LoRA (QLoRA) [7] techniques are used to reduce the needs of computational resources without affecting performance. The LLM4Models LLM is the result of this stage, and it can then be used to extract UML class diagram representations from arbitrary Java source code files.

2) *Inference Process*: LLM4Models is used to extract UML class diagrams from Java programs, but there are additional pre- and post-processing stages involved in this process, as recommended by Malyala *et al.* [8] to mitigate errors that can occur when utilizing machine learning techniques.

- **Input/Output Stages**: LLM4Models takes Java source code files as input, and produces UML class diagram representations as the output.
- **Pre-processing Stage**: The pre-processing stage involves removing comments and extra spaces from Java programs. Then, Java programs are tokenized to text strings that represent the elements of the Java program.
- **Large Language Models (LLMs)**: Our approach depends on a pre-trained LLM. The most powerful LLMs, such as GPT4 or Bard, need significant resources to utilise. We chose instead to fine-tune **Mistral-7B-v0.1**¹ as we have a small dataset and the performance is better than for Llama 2 13B [9].
- **Post-processing**: The post-processing stage is implemented to extract the desired representations with higher

accuracy than could be generated by directly utilizing the fine-tuned LLM.

- **Documentation/Program Representation**: To graphically display the generated UML class diagram, the output of LLM4Models is post-processed (in the previous stage), then a dot file that represents the generated UML class diagram is automatically built and translated to the Graphviz [10] framework, which then graphically draws the generated UML class diagram.

B. UML Class Diagram Extraction Rules

Different rules need to be applied to extract (i.e., to abstract) UML class diagram elements and relationships from Java source code files. These rules are used to generate the UML class diagrams corresponding to the sample Java programs in our dataset. The pairs of programs and class diagrams are then used to fine-tune the pre-trained LLM. These rules are explained as follows:

- **Identifying classes' names**: Classes' names can be recovered from Java source code syntactically, as they come after the **class** keyword. Each class is represented in the UML model by a rectangular shape.
 - **Identifying abstract and concrete classes**: Both abstract and concrete classes can be identified from the source code. The names of abstract classes are represented in *italic* format, in case the **abstract** keyword is found before the name of the class in the Java source code file, as opposed to the names of concrete classes, which are represented in normal text, as defined in UML standard notation.
 - **Identifying Inner Classes**: Java classes can have inner classes; thus, the pre-trained LLM is fine-tuned to recognise inner classes, which will be modeled in the same way as any other classes. An aggregation relationship from the outer class to the inner class is drawn in the generated UML class diagram.
- **Identifying interfaces**: Interface names can also be recovered from the Java source code file syntactically, where they come after the **interface** keyword. Interfaces are represented by using class rectangles with the **«interface»** stereotype shown above the interface name.

¹<https://huggingface.co/mistralai/Mistral-7B-v0.1>

- Identifying attributes: Classes in Java programs declare different types of attributes. Attribute names are modeled in underlined format when the attributes are static, i.e., with the `static` keyword occurring before the attribute names in the Java source code file, otherwise, they are modeled in normal text as defined in UML standard notation.
- Identifying methods and their parameters: Classes in Java programs declare methods and their parameters. Method names are modeled in underlined format when these methods are static, with the `static` keyword occurring before the method names in the Java source code file; otherwise, they are modeled in the normal text font as defined in UML standard notation. Also, methods are modeled in *italic* format when they are abstract methods. We aim to draw methods in three formats: methods' names only, methods' names with parameters' types, and methods' names with parameters' names, parameters' types, and return type.
- Identifying Data Types: The pre-trained LLM is fine-tuned to recover the data types of attributes, method parameters, local variables, and the return values of different methods from the Java source code files. The data types can be primitive types, inbuilt Java library classes, or user-defined classes.
- Identifying relationships between classes and interfaces: Different kinds of relationships can be found between classes and interfaces. While there is a lack of precise definitions for some relationships between classes, such as association, aggregation, and dependency [11], other relationships can be discovered straightforwardly, such as generalization and realization relationships. We opt to identify relationships between classes and interfaces as follows:
 - Generalization (Inheritance) relationship: A generalization relationship can be discovered syntactically from Java source code files, when the **extends** keyword is identified in the Java code.
 - Realization relationship: As in the generalization case, a realization relationship can be discovered syntactically from the Java source code files, in cases where the **implements** keyword is used to implement realization relationships in the Java programming language.
 - Association relationship: An association relationship can be inferred between two classes when the type of an attribute of one class is another class in the system. The association relationship can be a uni-directional or bi-directional relationship. The uni-directional association relationship exists when one class contains an attribute with the type of the other class, while the bi-directional association relationship is determined when the two classes each have an attribute of the respective other class type [12]. Any pair of uni-directional associations can be merged into one bi-directional association [13].
 - Aggregation relationship: An aggregation relationship is a special case of an association relationship. The pre-trained LLM is fine-tuned to generate the aggregation relationship if one class, the whole, contains an attribute of another class, the part. The whole class must define a field or an array field of the type of the part class and must create an instance of this field in the whole class. The aggregation relationship is also drawn if one class contains another class (the case of an inner class).
 - Dependency relationship: Finally, a dependency relationship is identified when the relationship between the two classes is quite weak. It arises when method parameters, local variables, or return values in a method of a class are of another class type [12], [14]. The pre-trained LLM is fine-tuned to recover dependency relationships in this manner.
- Identifying visibility of attributes and methods: The pre-trained LLM is fine-tuned to identify the visibility of attributes and methods as follows: assigning the symbol “+” for public attributes and methods, assigning the symbol “-” for private attributes and methods, and assigning the symbol “#” for protected attributes and methods.
- Identifying multiplicities: Multiplicities can be recovered by analyzing declared fields. If the field is an array/collection and a value is assigned to it, the multiplicity is set to “1..*”. The multiplicity is “0..*” if the field is an array/collection and is not assigned a value. If the field is not an array or collection, the multiplicity is “0..1” when no value is assigned to the field, otherwise it is “1”.
- Role names: A role name is the name of the attribute itself, and it is shown at the target association end of a uni-directional association relationship. When two uni-directional association relationships are merged, both role names are taken, resulting in a bi-directional association relationship with the role names at both ends.

III. ILLUSTRATIVE EXAMPLE

We use a simple example, Listing 1, containing different attributes, methods, and inheritance relationship to demonstrate how the proposed approach represents a UML class diagram.

Listing 1. An illustrative Java example.

```
import java.util.*;
abstract class AbstractSmartContract {
    public boolean checkSC(int sourceID, int targetID) {
        return sourceID != targetID;
    }
}
class Transaction extends AbstractSmartContract {
    private int sourceID;
    private int targetID;
    public int getSourceID() {
        return sourceID;
    }
    public int getTargetID() {
        return targetID;
    }
}
```

Two JSON specifications are produced by LLM4Models. The first represents the UML class diagram elements, while the second presents the relationships between classes and interfaces:

Listing 2. A JSON representation of Java program elements.

```
{
  "ClassInterface": "Class",
  "name": "AbstractSmartContract",
  "Visibility": "",
  "IsStatic": "",
  "IsAbstract": "abstract",
  "superclass": null,
  "interfaces": [],
  "variables": [],
  "methods": [
    {
      "name": "checkSC",
      "Visibility": "public",
      "IsStatic": "",
      "IsAbstract": "",
      "returnType": "boolean",
      "parameters": [
        {
          "name": "sourceID",
          "type": "int"
        },
        {
          "name": "targetID",
          "type": "int"
        }
      ]
    }
  ],
  "localVariables": []
},
{
  "ClassInterface": "Class",
  "name": "Transaction",
  "Visibility": "",
  "IsStatic": "",
  "IsAbstract": "",
  "superclass": "AbstractSmartContract",
  "interfaces": [],
  "variables": [
    {
      "name": "sourceID",
      "IsStatic": "",
      "IsAbstract": "",
      "type": "int",
      "Visibility": "private"
    },
    {
      "name": "targetID",
      "IsStatic": "",
      "IsAbstract": "",
      "type": "int",
      "Visibility": "private"
    }
  ],
  "methods": [
    {
      "name": "getSourceID",
      "Visibility": "public",
      "IsStatic": "",
      "IsAbstract": "",
      "returnType": "int",
      "parameters": [],
      "localVariables": []
    },
    {
      "name": "getTargetID",
      "Visibility": "public",

```

```

    "IsStatic": "",
    "IsAbstract": "",
    "returnType": "int",
    "parameters": [],
    "localVariables": []
  ]
}

```

Listing 3. A JSON representation of Java program relationships.

```
{
  "Relationship": "Inheritance",
  "Source": "Transaction",
  "Target": "AbstractSmartContract",
  "Multiplicity1": "",
  "Role1": "",
  "Multiplicity2": "",
  "Role2": "",
  "Flag": false
}
```

The output of LLM4Models is processed by a special program (the post-processing phase), then the dot format file required by the Graphviz tool is automatically generated and sent to the Graphviz tool to graphically represent the UML class diagram, as shown in Fig. 2.

IV. EVALUATION

The performance of LLM4Models is evaluated by using 10 randomly selected examples from the test set, where the dataset is split into training set, validation set, and test set.

Firstly, we measure the *accuracy* of the produced UML class diagram by calculating the proportion of source code elements and relationships represented correctly in the generated UML class diagram. $Accuracy(A) = \frac{TP}{TP+FN}$.

Where a true positive (TP) is an element correctly translated from Java code to a UML class diagram element, and a false negative (FN) is an element not translated or translated incorrectly.

Secondly, we measure the *consistency* by measuring the proportion of elements and relationships in the generated class diagram which are correctly derived from Java source code. $Consistency(C) = \frac{TP}{TP+FP}$.

Where a false positive (FP) is a UML element that appears in the generated UML class diagram elements, and that is not derived from source code.

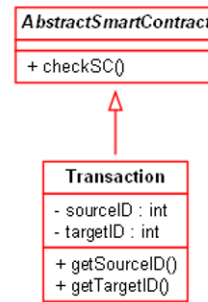


Fig. 2. The generated UML class diagram produced by LLM4Models.

A combined measure ($F1$) is used to balance between accuracy and consistency as: $F1 = \frac{2 \cdot A \cdot C}{A + C}$.

Accuracy, consistency, and $F1$ are also measured for the baseline LLM, and the results are compared to the results of LLM4Models, as shown in Table I.

TABLE I
EVALUATION OF ACCURACY/CONSISTENCY OF LLM4MODELS.

Model	Accuracy	Consistency	Combined
Baseline	50.68%	97.36%	66.66%
LLM4Models	99.3%	99.3%	99.3%

These results show that the fine-tuning process has produced a more effective LLM for Java program abstraction, which when combined with pre- and post-processing, achieves improved accuracy and $F1$ -score for the abstraction task compared to the baseline untrained LLM.

The training data, evaluation examples, and analysis results are in the repository <https://doi.org/10.5281/zenodo.14045263>.

V. RELATED WORK

Over the past two decades, reverse engineering techniques have been extensively researched, including model-driven approaches. Raibulet *et al.* [15] carried out a systematic literature review (SLR) up until 2017, identifying fifteen MDRE approaches. They examined different characteristics, including genericity, level of automation, and employed techniques to generate different models. Siala *et al.* [4] also performed an SLR covering the period from 2000 to 2023. The study identified 64 distinct MDRE approaches aimed at 14 different objectives. They found that the most targeted programming language is Java and the UML class diagram is the most commonly produced diagram from MDRE approaches. Methakullawat and Limpiyakorn [16] extract class diagrams from legacy PL/SQL code, while Sabir *et al.* [17] proposed an approach to extract both class diagrams and activity diagrams from Java source code files. Zhang H. [18] also extracts class and sequence diagrams from Java source code files. Nanthamornphong *et al.* [19] abstract class and sequence diagrams from Object-oriented Fortran code. Lano K. [20] and Lano *et al.* [21] approaches extract UML and OCL specifications from seven languages: Java, Python, C, JavaScript, VB, COBOL, and Pascal.

To our knowledge, the majority of these approaches use static techniques through source code analysis, and none of them use machine learning techniques, specifically LLMs, to extract different representations.

VI. CONCLUSION

We have defined a new MDRE approach termed LLM4Models for extracting UML class diagrams from Java source code files using LLMs. The extracted representations are graphed using the Graphviz tool. Both the illustrative example and evaluation section demonstrate the effectiveness of using LLMs in reverse engineering process. For future work, we aim to enhance the training database by adding more training samples to obtain better and more accurate results.

ACKNOWLEDGMENT

Hanan Siala acknowledges the financial support provided by the Libyan Ministry of Higher Education and Scientific Research and King's Computational Research, Engineering and Technology Environment (CREATE) <https://doi.org/10.18742/rnvf-m076>.

REFERENCES

- [1] S. Kent, "Model driven engineering," in *Integrated Formal Methods: Third International Conference, IFM 2002 Turku, Finland, May 15–18, 2002 Proceedings*, pp. 286–298, Springer, 2002.
- [2] OMG, "OMG Systems Modeling Language (UML), Version 2.5.1," 2017. <https://www.omg.org/spec/UML>.
- [3] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," 2023.
- [4] H. A. Siala, K. Lano, and H. Alfraihi, "Model-driven approaches for reverse engineering—a systematic literature review," *IEEE Access*, vol. 12, pp. 62558–62580, 2024.
- [5] Eclipse, "Eclipse agileuml project," 2022. <https://projects.eclipse.org/projects/modeling.agileuml>.
- [6] W. U. Ahmad, M. G. R. Tushar, S. Chakraborty, and K.-W. Chang, "Avatar: A parallel corpus for java-python program translation," *arXiv preprint arXiv:2108.11590*, 2021.
- [7] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [8] A. Malyala, K. Zhou, B. Ray, and S. Chakraborty, "On ml-based program translation: Perils and promises," 2023.
- [9] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. I. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.
- [10] G. D. Team, "Graphviz - graph visualization software," 2023. <https://www.graphviz.org/>.
- [11] Y.-G. Guéhéneuc, "A reverse engineering tool for precise class diagrams," in *Proceedings of the 2004 conference of the Centre for Advanced Studies on Collaborative research*, pp. 28–41, 2004.
- [12] S. Matzko, P. J. Clarke, T. H. Gibbs, B. A. Malloy, J. F. Power, and R. Monahan, *Reveal: A tool to reverse engineer class diagrams*. Australian Computer Society, Inc., 2002.
- [13] R. Kollmann and M. Gogolla, "Application of uml associations and their adornments in design recovery," in *Proceedings Eighth Working Conference on Reverse Engineering*, pp. 81–90, IEEE, 2001.
- [14] P. Tonella and A. Potrich, "Reverse engineering of the uml class diagram from c++ code in presence of weakly typed containers," in *Proceedings IEEE International Conference on Software Maintenance. ICSM 2001*, pp. 376–385, IEEE, 2001.
- [15] C. Raibulet, F. A. Fontana, and M. Zanoni, "Model-driven reverse engineering approaches: A systematic literature review," *Ieee Access*, vol. 5, pp. 14516–14542, JULY 2017.
- [16] N. Methakullawat and Y. Limpiyakorn, "Reengineering legacy code with model transformation," *International Journal of Software Engineering and Its Applications*, vol. 8, no. 3, pp. 97–110, 2014.
- [17] U. Sabir, F. Azam, S. U. Haq, M. W. Anwar, W. H. Butt, and A. Amjad, "A model driven reverse engineering framework for generating high level uml models from java source code," *IEEE access*, vol. 7, pp. 158931–158950, 2019.
- [18] H. Zhang, "An approach for extracting uml diagram from object-oriented program based on j2x," in *2016 International Forum on Mechanical, Control and Automation (IFMCA 2016)*, pp. 266–276, Atlantis Press, 2017.
- [19] A. Nanthamornphong, K. Morris, and S. Filippone, "Extracting uml class diagrams from object-oriented fortran: Formul," in *Proceedings of the 1st International Workshop on Software Engineering for High Performance Computing in Computational Science and Engineering*, pp. 9–16, 2013.
- [20] K. Lano, "Program translation using model-driven engineering," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pp. 362–363, 2022.
- [21] K. Lano, H. Haughton, Z. Yuan, and H. Alfraihi, "Program abstraction and re-engineering: an agile mde approach," *SAM/MODELS*, 2023.